# Autonomous Synthesis of Hard RTL Designs via Iterative Repair and Modular Decomposition

Anonymous Author(s)

## Abstract

Large language models can generate short register-transfer level (RTL) fragments, but benchmark-level hardware synthesis requires executable correctness: simulation success, protocol consistency, and golden-output agreement. ArchXBench exposes this gap through a reported hard-task frontier beginning at Level 4 under direct prompting. This paper evaluates whether verifier-grounded autonomous synthesis can cross that frontier without confusing model failures with benchmark-contract failures. Four conditions are compared on ArchXBench Levels 3–6: single-shot generation (C1), monolithic golden-feedback repair (C2g), decomposition-guided iterative repair (C4i), and testbench-localized decomposition (C4tl). C4i solves five Level-3 numerical kernels at 3/3 seeds where both C1 and C2g fail. C4tl solves all four core Level-4 designs, including 16-point FFT/IFFT, at 5/5 seeds. Golden-verified Level-5 and Level-6 solves are obtained on convolution, image-processing, and AES designs. Decomposition is not universally superior: C2g solves several Level-5 rows that decomposed methods do not. The central contribution is an executable-contract-aware synthesis methodology: original-benchmark solves, minimally repaired executable contracts, and held rows are separated before success is claimed. Code, prompts, logs, generated RTL, and result JSONs are prepared for anonymous artifact release.[1]

## CCS Concepts

• **Hardware** → **Hardware description languages and compilation**; **Electronic design automation**; • **Software and its engineering** → *Automatic programming*.

## Keywords

benchmark evaluation, CEGIS, hardware verification, LLM agents, RTL synthesis, Verilog generation

**ACM Reference Format:**
Anonymous Author(s). 2027. Autonomous Synthesis of Hard RTL Designs via Iterative Repair and Modular Decomposition. In *Proceedings of 32nd Asia and South Pacific Design Automation Conference (ASP-DAC '27).* ACM, New York, NY, USA, 6 pages.

## 1 Introduction

Autonomous code agents close executable loops: generate a candidate, run a checker, inspect failures, and repair. For software tasks, this loop has changed what counts as a meaningful benchmark result. For hardware design, the question is harder: can an autonomous agent synthesize RTL that survives simulation, protocol assumptions, and golden-output comparison?

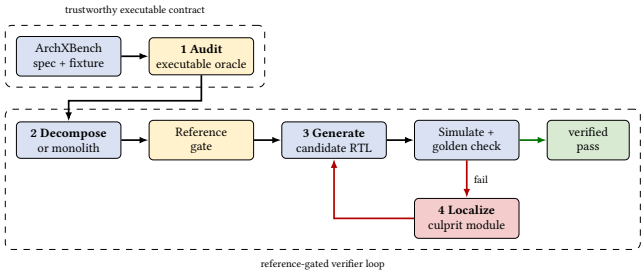


**Figure 1: C4tl pipeline: audit the executable contract, validate references, then repair only the localized culprit module.**

RTL correctness depends on bit widths, signedness, clocking, reset protocols, pipeline latency, and streaming handshakes. Hard RTL synthesis therefore tests whether autonomous agents can handle low-level, state-dependent, timing-sensitive design, not just surface-level code generation. The severity is practical: an RTL generator that compiles but mishandles latency, signed arithmetic, or output encoding can pass superficial checks while producing unusable hardware.

This paper studies the problem on ArchXBench, a benchmark suite for complex RTL synthesis [6]. Unlike VerilogEval [3] and RTLLM [4], which target smaller designs, ArchXBench spans arithmetic kernels, FFTs, filters, image processing, matrix multiplication, and cryptography across six difficulty levels. The ArchXBench paper reports a direct-generation frontier beginning at Level 4: lower levels are broadly reachable, while hard Level-4 rows such as FFT/IFFT expose near-zero zero-shot pass@5 behavior. If an autonomous synthesis loop can solve rows beyond it, the evaluation question changes from "can an LLM emit a complete design in one shot?" to "can a verifier-grounded agent converge to correct RTL?"

The central question is: *can verifier-grounded autonomous synthesis solve hard RTL benchmark tasks that direct generation does not solve, while maintaining a clear distinction between model failures and benchmark-contract failures?* The answer is yes, but not through a single universally dominant architecture. C4i solves five Level-3 numerical kernels at 3/3 seeds where C1 and C2g both fail. C4tl solves all four core Level-4 designs at 5/5 seeds, including the 16-point FFT and IFFT. Golden-verified solves extend to Level-5 convolution and image processing and to Level-6 AES encryption and decryption. At the same time, C2g is a strong baseline: it solves several Level-5 rows and multiple repaired-contract designs that decomposed methods do not. The contribution is not a claim that decomposition dominates monolithic repair. It is evidence that verifier-grounded synthesis, in both monolithic and decomposed forms, can cross parts of the reported hard-task frontier, and that benchmark contracts must be audited before either successes or failures are interpreted.

This paper makes three contributions:

[1] Repository URL withheld for double-blind review.

(1) The proposed work introduces an executable-contract-aware evaluation discipline for autonomous RTL synthesis: released contracts, minimally repaired executable contracts, and held rows are separated before positive claims are made.
(2) The proposed work gives a verifier-grounded synthesis algorithm that instantiates direct generation, monolithic repair, decomposition-guided repair, and testbench-localized repair as controlled ablations of feedback, decomposition, reference validation, and localization.
(3) The proposed work demonstrates hard ArchXBench solves beyond the reported direct-generation frontier, including robust Level-4 original-contract FFT/IFFT solves and golden-verified Level-5/Level-6 solves.

## 2 Related Work

### 2.1 LLM-Based RTL Generation

VerilogEval and RTLLM made Verilog generation measurable with automatic checking [3, 4]. Subsequent systems add feedback, planning, workflow search, or evolutionary search for RTL generation and repair [1, 5, 7, 8, 11, 12]. Recent benchmark and verification work further shows that flawed executable contracts and weak oracles can distort RTL evaluation [2, 9, 10]. Table 1 summarizes where the present study sits.

Our work evaluates on ArchXBench Levels 3–6, including rows at and above the reported Level-4 frontier. It separates original benchmark contracts from repaired executable contracts so that model failures are not confused with stale testbenches, file-format mismatches, or inconsistent golden comparators.

### 2.2 Verifier-Grounded Synthesis

Counterexample-guided inductive synthesis (CEGIS) uses a verifier to reject incorrect candidates and guide the next attempt. LLM-based repair loops instantiate a related pattern: generate code, run tests, inspect errors, and repair. For RTL, verifier feedback is heterogeneous: failures arise from syntax, elaboration, simulation, timing, protocol mismatch, file-output mismatch, or incorrect golden references. The present pipeline treats simulator and golden-output feedback as first-class signals and audits the benchmark contract when executable artifacts disagree.

## 3 Problem Setting

Each ArchXBench design provides a natural-language specification and executable assets: Verilog testbenches, input files, output files, and comparison scripts. An autonomous synthesis condition receives the task and emits RTL. For self-checking designs, a run succeeds only when the official testbench passes all checks. For file-output designs, simulation completion is not sufficient; the generated output must match the golden output element by element.

Three evaluation classes are distinguished:

- **Original-contract results:** runs against the released benchmark as-is.
- **Repaired-contract results:** runs against a copied fixture with minimally repaired infrastructure inconsistencies.
- **Held or excluded rows:** tasks where the released contract remains too ambiguous for a credible positive result.

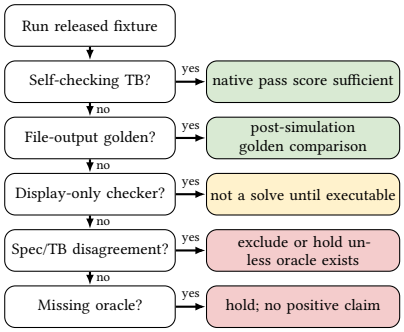


**Figure 2: Executable-contract classification. Positive results are claimed only when the released or repaired fixture has an executable oracle.**

## 4 Method

### 4.1 Evaluated Conditions

The condition grid below summarizes the operational differences among the four conditions. All conditions receive the same specification, module interface, and released testbench. The difference is how much executable feedback enters the loop and whether the design is treated as a monolith or decomposed into modules. Thus the four conditions are also ablations: C1 removes feedback, C2g removes decomposition, C4i adds decomposition without localized repair, and C4tl adds a strict reference gate plus localization.

**Condition grid.** Operational differences among synthesis conditions. "Ref. gate" indicates whether the decomposition's reference composition must pass the system testbench before synthesis proceeds. "Local repair" describes repair granularity.

| Cond. | Decomp. | Ref. gate | Golden fb. | Local repair | Budget |
|---|---|---|---|---|---|
| C1 | no | – | no | – | 5 shots |
| C2g | no | – | yes | whole design | 30 rounds |
| C4i | yes | sanity | yes | per module | ≈28 rds |
| C4tl | yes | strict | yes | culprit only | 3+≈28 rds |

**C1: direct generation.** The model receives the specification and testbench excerpt and emits five independent RTL candidates. The highest-scoring candidate is retained; no repair feedback is used.

**C2g: monolithic golden-feedback repair.** The model emits a complete top module, then iteratively repairs it using compile errors, self-check failures, and golden-output mismatches with expected-versus-actual indices. The loop terminates when all checks pass or after 30 rounds; long conversations are pruned to the first two and last sixteen messages. This is a deliberately strong baseline because it uses the same executable oracle as decomposed methods without paying module-integration risk.

**C4i: decomposition-guided iterative repair.** C4i decomposes the task into 3–7 submodules with explicit interfaces, a top-level wrapper, and reference implementations. After a reference-composition sanity check, submodules are solved in fixed order. During submodule repair, unsolved neighbors are held at their references, isolating bugs to the target module; each of $n$ submodules receives $\lfloor 28/n \rfloor$ repair rounds.

**C4tl: testbench-localized decomposed repair.** C4tl uses the same decomposition structure as C4i but adds two mechanisms: a strict reference gate and trace-lifted fault localization.

**Table 1: Positioning relative to recent LLM-aided RTL work. The key distinction is a hard-ArchXBench study that combines verifier-grounded synthesis with explicit original-contract versus repaired-contract accounting.**

| Line of work | Examples | Main strength | Gap addressed here |
|---|---|---|---|
| RTL benchmarks | VerilogEval, RTLLM | Automatic functional checks | Harder ArchXBench L3–L6 rows |
| Agentic RTL generation | AutoChip, VerilogCoder, AutoVeriFix, Spec2RTL | Feedback, planning, tracing | Original vs repaired contract accounting |
| Workflow/search | VFlow, REvolution | Workflow and population search | Hard-frontier executable synthesis study |
| Benchmark/oracle validity | RTL-BenchMT, FormalRTL, RTL-BenchLS | Maintenance, formal oracles | Contract audit tied to synthesis claims |
| This work | C1/C2g/C4i/C4tl on ArchXBench | Golden repair, decomposition, localization | Cross frontier while separating held rows |

**Table 2: Executable-contract-aware verifier-grounded RTL synthesis. The condition parameter selects C1, C2g, C4i, or C4tl by enabling feedback, decomposition, reference gating, and localization.**

| | |
|---|---|
| **Input** | Specification $S$, released fixture $B$, condition $c$, seed $s$, budget $T$ |
| **Output** | SOLVED, UNSOLVED, or HELD, with saved artifact record |
| **Audit** | Classify $B$ using Fig. 2. If no executable oracle can be recovered, return HELD. |
| **Decompose** | If $c$ uses decomposition, request interfaces, top wrapper, and reference modules $R_1, \ldots, R_n$; otherwise set $n = 1$ and target the whole design. |
| **Gate** | When enabled, simulate the composed references against $B$; retry with failure feedback and abort if no valid reference composition is found. |
| **Generate** | Produce candidate RTL modules $M_1, \ldots, M_n$ from $S$, $B$, references, and seed $s$. |
| **Verify** | For each round $t \le T$, simulate the composed candidate and compute pass/golden score. If all checks pass, save RTL and return SOLVED. |
| **Localize** | Select a repair target: C2g selects the whole design; C4i selects the next module in fixed order; C4tl swaps each $M_i$ with $R_i$ and selects the module with the largest system-score improvement. |
| **Repair** | Repair only the selected target using compile errors, failing indices, expected/actual values, swap scores, and the relevant reference; then repeat verification. |
| **Exit** | Save the final artifact and return UNSOLVED. |

The reference gate requires the composed reference implementation to pass the system-level testbench before synthesis begins; failed references are retried up to three times, otherwise the run aborts. After candidate generation, C4tl localizes a failure by swapping each candidate submodule with its validated reference while all other candidates remain fixed. Let $M$ denote the current composed candidate and $M^{(i \leftarrow R_i)}$ denote replacing only module $i$ with its validated reference. For executable score function $q(\cdot)$, C4tl computes

$$\Delta_i = q(M^{(i \leftarrow R_i)}) - q(M), \qquad i^\star = \arg\max_i \Delta_i.$$

The selected module $M_{i^\star}$ is repaired using simulator output, golden mismatches, swap scores, and the reference implementation. The prompt asks the model to trace the first failing datapath, compute expected intermediate values, identify the root cause, and then fix.

The localization mechanism is the central algorithmic distinction. C4i repairs modules in a fixed order; C4tl uses system-level evidence to identify which module is most implicated. For tightly coupled pipelines such as the 16-point FFT, this targeted approach avoids diffuse repair of modules that may not be at fault. The localization step is heuristic rather than a correctness guarantee. It is strongest when the reference composition is valid and one module dominates the observed failure. It can be misled by coupled multi-module bugs, bad decompositions, or errors whose effects are masked until multiple modules are replaced together.

## 4.2 Artifact and Verification Discipline

Each run is recorded under a repository artifact tree with a structured result file containing the design, condition, model, seed, pass counts, golden counts, LLM-call count, wall time, and generated RTL. File-output designs require strict golden verification: the generated output is compared element by element against the golden reference with a tolerance of ±1 on integer values. Native simulation completion alone is treated as diagnostic evidence, not as a solve.

# 5 Experimental Setup

## 5.1 Benchmark Scope

The evaluation covers ArchXBench Levels 3–6: iterative arithmetic, FFT/IFFT, floating-point pipelines, FIR filters, image-processing kernels, matrix multiplication, and AES. Lower levels are excluded because the ArchXBench paper reports that current models already solve them. Execution uses repository scripts with Python orchestration, Icarus Verilog (`iverilog`), and the VVP runtime (`vvp`); each candidate is compiled before simulation and self-checking or golden-output comparison.

## 5.2 Seeds, Model, and Metrics

Main tables use seeds 42, 123, and 456. Two additional seeds (789, 1024) provide robustness evidence for C4tl Level-4 rows. These are fixed seeds chosen for reproducibility, not tuned seeds. C1 uses five seeds for the two borderline Level-3 floating-point rows, `fp_adder`

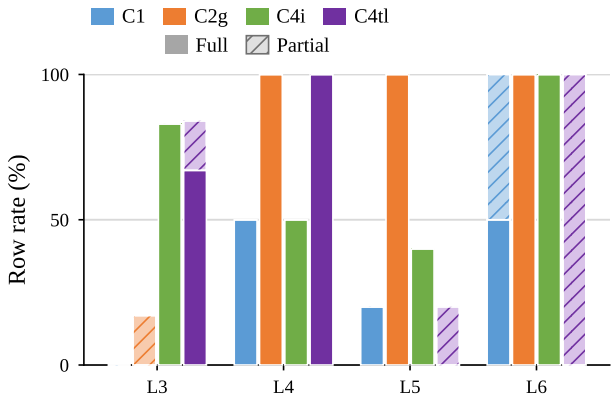


**Figure 3: Original-contract solve rate by level. Colors encode synthesis condition. Solid bars count rows solved on all main seeds; hatched segments count partial rows where at least one seed passes but not all main seeds pass.**

and `fp_multiplier`, to give a stricter direct-generation baseline estimate. The primary model is GPT-5.5 as recorded in the repository artifacts.

For run $r$, let $p_r/t_r$ denote native self-checking passes and $g_r/h_r$ denote golden-output matches. The executable score is

$$q(r) = \begin{cases} g_r/h_r, & h_r > 0, \\ p_r/t_r, & h_r = 0 \text{ and } t_r > 0, \\ 0, & \text{otherwise.} \end{cases}$$

Run $r$ solves iff $q(r) = 1$. For design $d$, condition $c$, and seed set $S$, full-row and partial-row indicators are

$$F_{d,c} = \prod_{s \in S} \mathbf{1}[q(d,c,s) = 1], \qquad P_{d,c} = \mathbf{1}\left[\max_{s \in S} q(d,c,s) = 1\right] - F_{d,c}.$$

Figure 3 reports these design-row rates by level.

All methods run as executable synthesis loops; the method budgets are summarized in the condition grid in Section 4.1. The Level-4 FFT solves in roughly 5 minutes and 6 LLM calls, while harder designs such as `harris_corner_detection` require 10.7 calls and 9.4 minutes on average. Failed decomposed runs typically consume the full repair budget: failed C4i/C4tl-style rows in the FFT/IFFT diagnostics use about 26–29 LLM calls and can take roughly 20–45 minutes on the local runner.

## 6 Crossing the Original-Contract Frontier

Table 3 reports the main original-contract results.

**Level 3.** C4i provides the strongest method-value evidence: it solves five numerical kernels at 3/3 seeds where C1 scores 0/3–0/5 and C2g scores 0/3–1/5. C4tl matches C4i on four of five rows but solves `gauss_siedel` at only 1/3. For context, C4i decomposes `gauss_siedel` into three submodules (reciprocal unit, iteration step, solution selector), allowing each to be solved and tested in isolation. The negative result on `newton_raphson_polynomial` is an original-checker ceiling: three of its 100 assertions are structurally unsatisfiable, capping the achievable score at 97/100. After removing only those three unsatisfiable residual assertions in the repaired-contract fixture, C2g solves all three seeds (Table 5), confirming that the design itself is within model capability.

**Level 4.** C4tl solves all four core Level-4 rows at 3/3 main seeds and 2/2 robustness seeds, for 5/5 total. The 16-point FFT and IFFT rows are the strongest hard-frontier evidence because they are self-checking original-contract solves with no benchmark repair. C4tl decomposes `fft_16pt_iterative` into four submodules (bit-reverse mapper, twiddle ROM, butterfly unit, output scaler) and uses trace-lifted localization to identify which module is at fault after each simulation failure. C2g also solves the FFT/IFFT rows at 3/3; C4i does not. C2g avoids the module-order problem by repairing the design monolithically; C4tl retains modular structure but uses swap-based localization to avoid spending repair budget on modules that already compose correctly. C4i's fixed sequential repair order can spend iterations on modules that already compose correctly while the actual fault remains in a later module; C4tl's swap-based localization directly identifies the culprit before repair. The pipeline rows (`fp_adder_pipeline`, `fp_mult_pipeline`) are solved by all conditions.

**Levels 5–6.** C4i obtains 3/3 golden-verified solves on `conv1d`, `harris_corner_detection`, and both AES rows. C4i decomposes AES encryption into six submodules (S-box, SubBytes, ShiftRows, MixColumns, round function, key expansion), each implemented and tested against the system testbench with other modules held at their references. Several Level-5 rows are solved only by C2g: `conv2d` (4,096/4,096), `dct_idct_8pt_pipelined` (16/16), and `unsharp_mask` (65,536/65,536). These rows demonstrate that monolithic repair is the stronger strategy when the design is compact enough for whole-design feedback or when the primary difficulty is a global convention such as file format or fixed-point encoding.

The results are not presented as a universal decomposition win. C2g is a serious baseline: it matches or outperforms decomposition on 8 of 17 original-contract rows. The most accurate interpretation is that decomposition provides value on specific hard rows and gives a structured synthesis process, while monolithic golden-feedback repair is often sufficient and sometimes better.

**Model generality.** To test whether the hard-frontier result is specific to `gpt-5.5`, Table 4 repeats the strongest original-contract rows with Claude Sonnet 5. The original Level-4 FFT/IFFT rows solve on all three seeds under both C2g and C4tl, and the Level-6 AES rows solve on all three seeds under C2g with golden verification. Sonnet 5 does not solve AES under C4tl because the reference decomposition fails before localized repair can begin. For AES encryption, Sonnet 5 produces decompositions whose reference compositions fail the original system testbench; for AES decryption, it does not produce a usable reference composition within the generation budget. This indicates model-dependent sensitivity in the decomposition gate, while monolithic golden-feedback repair remains robust on the same AES rows.

**Case study: original Level-4 FFT.** `fft_16pt_iterative` is a self-checking original-contract row, so no benchmark repair is involved. C1 fails on all three main seeds, while C2g and C4tl solve all three. The row therefore isolates the central mechanism: executable feedback can turn a direct-generation failure into a verified RTL solve without changing the released checker. In one robustness seed, C4tl localizes a 9/33 failure to the butterfly while

**Table 3: Original-contract result heatmap on ArchXBench. Green cells solve all main seeds; yellow cells are partial; red cells fail. GV marks golden-verified file-output rows. Level-4 C4tl rows additionally solve robustness seeds 789 and 1024, giving 5/5 total.**

| Level | Design | C1 | C2g | C4i | C4tl | Evidence |
|---|---|---|---|---|---|---|
| L3 | fp_adder | 0/5 | 1/5 | **3/3** | **3/3** | decomposition wins |
| L3 | fp_multiplier | 0/5 | 0/3 | **3/3** | **3/3** | decomposition wins |
| L3 | gauss_siedel | 0/3 | 0/3 | **3/3** | 1/3 | C4i strongest |
| L3 | gradient_descent | 0/3 | 0/3 | **3/3** | **3/3** | decomposition wins |
| L3 | newton_raphson_sqrt | 0/3 | 0/3 | **3/3** | **3/3** | decomposition wins |
| L3 | newton_raphson_polynomial | 0/3 | 0/3 | 0/3 | 0/3 | checker ceiling at 97/100 |
| L4 | fft_16pt_iterative | 0/3 | **3/3** | 0/3 | **3/3** | hard-row solve; 5/5 |
| L4 | ifft_16pt_iterative | 0/3 | **3/3** | 0/3 | **3/3** | hard-row solve; 5/5 |
| L4 | fp_adder_pipeline | **3/3** | **3/3** | **3/3** | **3/3** | all methods solve; 5/5 |
| L4 | fp_mult_pipeline | **3/3** | **3/3** | **3/3** | **3/3** | all methods solve; 5/5 |
| L5 | conv1d | **3/3** | **3/3** | **3/3** | 1/3 | GV |
| L5 | conv2d | 0/3 | **3/3** | 0/3 | 0/3 | GV; C2g strongest |
| L5 | dct_idct_8pt_pipelined | 0/3 | **3/3** | 0/3 | 0/3 | GV; C2g strongest |
| L5 | harris_corner_detection | 0/3 | **3/3** | **3/3** | 0/3 | GV |
| L5 | unsharp_mask | 0/3 | **3/3** | 0/3 | 0/3 | GV; C2g strongest |
| L6 | aes_encryption | **3/3** | **3/3** | **3/3** | 2/3 | GV |
| L6 | aes_decryption | 1/3 | **3/3** | **3/3** | 1/3 | GV; repair improves C1 |

**Table 4: Second-model validation with Claude Sonnet 5 on selected hard-frontier rows. GV denotes golden-verified file-output rows.**

| Design | C2g | C4tl |
|---|---|---|
| fft_16pt_iterative | **3/3** | **3/3** |
| ifft_16pt_iterative | **3/3** | **3/3** |
| aes_encryption | **3/3** GV | 0/3 GV |
| aes_decryption | **3/3** GV | 0/3 GV |

the twiddle ROM, index generator, and bit-reversal modules each pass in isolation; one repair then passes all 33 checks. The repaired RTL explicitly implements signed Q1.15 complex rotation.

**Listing 1.** Generated RTL excerpt from the Level-4 FFT butterfly repair.

```
assign prod_rr = b_real * tw_cos_q15;
assign prod_is = b_imag * tw_sin_q15;
assign prod_ic = b_imag * tw_cos_q15;
assign prod_rs = b_real * tw_sin_q15;
assign rot_real_w = (prod_rr - prod_is + ROUND) >>> 15;
assign rot_imag_w = (prod_ic + prod_rs + ROUND) >>> 15;
assign y0_real = a_real + rot_real;  assign y1_real = a_real - rot_real;
assign y0_imag = a_imag + rot_imag;  assign y1_imag = a_imag - rot_imag;
```

## 7 Executable-Contract Audit

Hard RTL benchmarks are useful only when their executable contracts reflect the intended task. During evaluation, several rows were found to contain inconsistent specifications, stale comparators, display-only checkers, or output-schema mismatches. The released benchmark is not overwritten. Repaired fixtures are kept under a separate artifact root and reported separately.

The audit follows a fixed protocol. First, the released fixture is run unchanged. Second, the checker is classified (self-checking, file-output with golden, display-only, stale, specification-inconsistent, or missing-oracle). Third, file-output rows are judged only by post-simulation golden comparison. Fourth, infrastructure repairs are applied only in copied fixtures and only when the intended oracle is recoverable from released assets. Fifth, unresolved rows are held rather than converted into positive results. The repair rule is intentionally narrow: a repaired fixture may make an existing oracle executable or repair file-format infrastructure, but it may not change the semantic task, introduce hidden target behavior, or move a row into the original-contract table.

**Table 5.** Repaired-contract results. These are separate from original ArchXBench claims.

| Design | C2g | C4i | C4tl |
|---|---|---|---|
| conv_3d | 3/3 | 2/3 | 0/3 |
| multich_conv2d | 3/3 | 3/3 | 3/3 |
| quantized_matmul | 3/3 | 3/3 | 0/3 |
| fp_band_pass_fir | 3/3 | 0/1 | 0/1 |
| fp_high_pass_fir | 3/3 | 0/1 | 0/1 |
| newton_raphson_polynomial | 3/3 | 1/3 | 1/3 |
| systolic_gemm | 0/3 | 0/3 | 0/3 |

The audit produces both positive and negative evidence. Repairing the executable contract for `multich_conv2d` and `quantized_matmul` makes the intended task measurable and solvable. By contrast, converting the display-only `systolic_gemm` checker into executable checks does not produce a win: all methods remain at 0/3. This is a genuine remaining capability boundary, not a benchmark-repair success.

Several rows remain held or excluded. The L4 FIR family is excluded because the specification and testbench disagree on filter coefficients. The L6 `fp_low_pass_fir` is held because the released files do not expose an explicit coefficient oracle. The L6 `fft_streaming_64pt` is excluded because the released input/output contract has unresolved schema and numeric-encoding ambiguities.

## 8 Discussion

### 8.1 When Decomposition Helps and When It Does Not

Decomposition is most effective when a design factors naturally into modules and when failure localization reduces the repair burden. The Level-3 numerical kernels illustrate the strongest decomposition-over-monolith wins, while the Level-4 FFT/IFFT

rows show a different effect: both C2g and C4tl cross the direct-generation frontier, but fixed-order C4i fails. C4i fails on the FFT/IFFT rows where C4tl succeeds. One explanation is that C4i's fixed per-module repair order is less effective for tightly coupled pipeline stages; C4tl's trace-lifted localization identifies the culprit module and avoids diffuse iteration on modules that are not at fault. A randomized-order C4i ablation improves FFT/IFFT from 0/6 to 3/6 solved seeds, but remains below C4tl's 6/6 main-seed success, supporting the interpretation that order matters but targeted localization is more reliable than random scheduling alone.

By contrast, C2g solves several Level-5 rows that neither C4i nor C4tl can solve. This happens when the design is compact enough for whole-design repair, when module boundaries introduce integration risk, or when the primary difficulty is a global convention such as file format or fixed-point encoding. The paper's claim is therefore conservative: decomposition is a powerful synthesis strategy for some hard rows, but monolithic repair remains strong and should be treated as a primary baseline.

### 8.2 Why Benchmark Contracts Matter

LLM-aided RTL synthesis papers increasingly rely on executable benchmarks. If the executable contract is ambiguous, a model can fail for the wrong reason or pass the wrong task; separating original-contract and repaired-contract results is therefore a prerequisite for credible evaluation.

## 9 Limitations

The evaluated methods do not establish universal dominance over monolithic repair; C2g should be treated as a primary baseline. Historical score-only diagnostics are kept in the artifact inventory for audit completeness but are not used for headline claims. Repaired-contract evaluation depends on judgment about what constitutes an infrastructure repair rather than a semantic benchmark change; this risk is mitigated by keeping repaired fixtures separate and excluding unresolved rows. The synthesis conditions use fixed prompt templates; sensitivity to prompt wording was not systematically evaluated and remains a source of uncontrolled variation. The empirical study is concentrated on RTL synthesis; transfer to other executable synthesis domains is not established here.

## 10 Conclusion

This study shows that the hard RTL frontier is not fixed by direct generation alone. On ArchXBench, verifier-grounded repair crosses original-contract Level-4 FFT/IFFT rows and obtains golden-verified Level-5/Level-6 solves, while repaired-contract experiments expose additional benchmark tasks that were previously unmeasurable. The results also show why the comparison must be disciplined: monolithic golden-feedback repair is a strong baseline, decomposition helps selectively, and ambiguous executable contracts can turn both failures and successes into misleading evidence.

Hard autonomous RTL synthesis therefore requires all three pieces together: structured generation, executable feedback, and trustworthy benchmark contracts.

## References


[1] Chia-Tung Ho, Haoxing Ren, and Brucek Khailany. 2025. VerilogCoder: Autonomous Verilog Coding Agents with Graph-based Planning and Abstract Syntax Tree (AST)-based Waveform Tracing Tool. In *Proceedings of the AAAI Conference on Artificial Intelligence.*

[2] Kezhi Li, Min Li, Xiangyu Wen, Shibo Zhao, Jieying Wu, Junhua Huang, and Qiang Xu. 2026. FormalRTL: Verified RTL Synthesis at Scale. *arXiv preprint arXiv:2603.08738* (2026).

[3] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design.*

[4] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2023. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. *arXiv preprint arXiv:2308.05345* (2023).

[5] Kyungjun Min, Kyumin Cho, Junhwan Jang, and Seokhyeong Kang. 2026. REvolution: An Evolutionary Framework for RTL Generation Driven by Large Language Models. In *Proceedings of the Asia and South Pacific Design Automation Conference.*

[6] Suresh Purini, Siddhant Garg, Mudit Gaur, Sankalp Bhat, Sohan Mupparapu, and Arun Ravindran. 2025. ArchXBench: A Complex Digital Systems Benchmark Suite for LLM Driven RTL Synthesis. In *Proceedings of the 7th ACM/IEEE Symposium on Machine Learning for CAD.* 1–10. doi:10.1109/MLCAD65511.2025.11189156

[7] Yan Tan, Xiangchen Meng, Zijun Jiang, and Yangdi Lyu. 2026. AutoVeriFix: Automatically Correcting Errors and Enhancing Functional Correctness in LLM-Generated Verilog Code. In *Proceedings of the Asia and South Pacific Design Automation Conference.*

[8] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2024. AutoChip: Automating HDL Generation Using LLM Feedback. In *Proceedings of the ACM/IEEE Design Automation Conference.*

[9] Jing Wang, Shang Liu, Wenji Fang, Yuchao Wu, Yugao Zhu, and Zhiyao Xie. 2026. RTL-BenchLS: A Large-Scale Benchmark for RTL Reasoning and Generation with Large Language Models. *arXiv preprint arXiv:2606.08976* (2026).

[10] Jing Wang, Shang Liu, Hangan Zhou, and Zhiyao Xie. 2026. RTL-BenchMT: Dynamic Maintenance of RTL Generation Benchmark Through Agent-Assisted Analysis and Revision. In *Proceedings of the ACM/IEEE Design Automation Conference.* doi:10.1145/3770743.3804053

[11] Yangbo Wei, Zhen Huang, Huang Li, Wei W. Xing, Ting-Jung Lin, and Lei He. 2026. VFlow: Discovering Optimal Agentic Workflows for Verilog Generation. In *Proceedings of the Asia and South Pacific Design Automation Conference.*

[12] Zhongzhi Yu, Mingjie Liu, Michael Zimmer, Yingyan Celine Lin, Yong Liu, and Haoxing Ren. 2025. Spec2RTL-Agent: Automated Hardware Code Generation from Complex Specifications Using LLM Agent Systems. *arXiv preprint arXiv:2506.13905* (2025).